

Copy elision for direct-initialization with a conversion function ([Core issue 2327](#))

Document number: P2828R1

Date: May 11, 2023

Audience: CWG

Reply to: Brian Bi (bbi5291@gmail.com)

With thanks to: Richard Smith

Revision history

R1

- Added proposed wording
- Added Example 10

Copy elision in the Big 4

Consider the example given in the issue report:

```
// Example 1
struct Cat {};
struct Dog { operator Cat(); };

Dog d;
Cat c(d);
```

Recent versions of Clang, GCC, MSVC, and NVC++ (which uses an EDG front end) all implement copy elision, and accept the code even if `Cat`'s move constructor is explicitly deleted. However, each implementation has taken a different approach in order to achieve this result. Comparison of the approaches reveals a surprisingly large design space for copy elision for direct-initialization using a conversion function. Even if (as will be proposed later in this paper) CWG adopts the most conservative possible approach as the resolution for issue 2327, I believe there is value in discussing the strengths and weaknesses of the various different approaches currently found “in the wild” in case CWG (or EWG) sees fit to expand the scope of copy elision in the future.

(As an obvious disclaimer, I do not have access to the EDG or MSVC source code, so any statements I make about their copy elision logic are mere educated guesses. I am also not affiliated with, nor authorized to make any official statements regarding, any of the four implementations surveyed herein.)

EDG appears to take the simplest and most conservative possible approach: perform overload resolution exactly as the standard currently requires, and if the copy or move constructor of `Cat` is selected to perform the direct-initialization, and a temporary of type `cv Cat` would be materialized in order to bind the parameter of the constructor to the prvalue result of calling a conversion function, then just use that prvalue to directly construct the object being initialized instead of creating a temporary. Since I don't have access to the EDG source code, I can't be exactly sure that this is what it's doing, but an examination of various cases on which EDG diverges from Clang, GCC, and MSVC showed that this rule would explain the divergence every time.

Clang's approach, as explained to me by Richard Smith, involves considering both constructors and conversion functions as candidates for the direct-initialization. In the above example, the three candidate functions for the initialization are therefore the copy and move constructor of `Cat` and the conversion function in `Dog`. In order to call

either constructor of `Cat`, a user-defined conversion (namely, `Dog::operator Cat`) must be invoked on the initializer `d`. But `operator Cat` itself is also a candidate, and it only requires a standard conversion sequence (namely the reference binding of the implicit object parameter). Consequently, it wins the overload resolution and is called to initialize `c`. Clang's approach represents the most aggressive modification to the overload resolution rules, and thus sits on the opposite end of the spectrum from EDG.

GCC's approach, which I discovered by simply reading the source code, has characteristics of both Clang's approach and EDG's approach. It first enumerates the candidate constructors as the standard currently requires, but, when determining the overload resolution priority of a copy or move constructor whose reference parameter would bind to the prvalue result of a conversion operator to type `cv Cat`, that constructor is considered to be replaced by the conversion function itself (which increases its overload resolution priority for the reasons explained in the previous paragraph).

Finally, the MSVC approach is very conservative (similar to EDG) but does appear to diverge from it slightly. The divergence is mysterious and I'm not able to come up with a coherent hypothesis as to what logic it uses. This will be discussed later in the paper.

In all cases, the only relevant conversion functions are the ones with return type exactly `cv Cat`; not a derived class, and not a reference. That's because you can't perform copy/move elision unless the conversion function returns a prvalue of type exactly `cv Cat`.

Known divergences

In this section I discuss the known divergences between the four implementations. Because of the conservative nature of EDG's approach, we can assume that it doesn't break any currently valid code (other than in possible cases where SFINAE causes a different overload to be chosen when a previously ill-formed construct becomes well-formed); at the same time, EDG's approach also never “improves” overload resolution (*i.e.* it cannot make any currently ambiguous cases unambiguous). Therefore, I'll treat EDG's approach as if it's the status quo when discussing the pros and cons of the other three approaches.

Disambiguating in favor of move constructors when multiple conversion operators are present

This is the most common type of implementation divergence noticed by Stack Overflow users. Its most common incarnation boils down to:

```
// Example 2
struct X {
    X(int);
    // X(X&&); // implicitly declared
};

struct Y {
    operator X();
    operator int();
};

X x(Y{});
```

The status quo is that this is ambiguous because the candidates `X::X(int)` and `X::X(X&&)` each require a different user-defined conversion function (`Y::operator int` and `Y::operator X`), respectively. Clang and GCC exhibit improved behavior in such cases: Clang by treating `operator X` as a candidate, and GCC by replacing `X::X(X&&)` by `operator X` when comparing it against `X::X(int)`.

In the February 2023 WG21 meeting in Issaquah, the consensus in CWG was that code like the above ought to be well-formed, and that an approach similar to the Clang approach should be pursued in order to make it so. However, when the issue was discussed again at the March 3, 2023 telecon, concerns were expressed about overly ambitious

revisions to the overload resolution rules (see particularly Example 4 below). The consensus was that the benefits to making Example 2 work are outweighed by the costs of changing the currently mandated behavior of Example 4 to a different (but also well-formed) behavior.

This divergence also arises in the context of standard library classes that have multiple one-argument constructors:

```
// Example 3
#include <string>

struct X {
    template <typename T>
    operator T();
};

std::string s(X{});
// string(string&&)?
// string(const char*)?
// string(const allocator&)?
// string(initializer_list<char>)?
// string(nullptr)?
```

Here, virtually anyone but a language lawyer would expect the conversion operator to be called to convert to `std::string`. However, having an unconstrained conversion function template as in this example is not a particularly compelling use case. The consensus of CWG in the March 3, 2023 telecon was again that making this example work should not come at the cost of changing the behavior of Example 4.

Preference of conversion functions over constructors that would use a standard conversion

In the previous section we discussed some cases where the Clang/GCC approach picks the “right” constructor/conversion operator pair instead of leaving it ambiguous. However, there are also currently valid cases where the Clang/GCC approach *changes* the behavior in possibly surprising ways: a conversion function that is suitable for elision will be preferred over most constructors even if the implicit conversion sequence for the constructor is a standard conversion sequence. For example:

```
// Example 4
struct Dog;

struct Cat {
    Cat(const Dog&);
};

struct Dog {
    operator Cat();
};

Cat cat(Dog{});
```

In current C++ and in EDG and MSVC, this will call the converting constructor. In Clang/GCC, this will call `operator Cat` because when `operator Cat` is a candidate, its implicit conversion sequence is the binding of the implicit object parameter of type `Dog&`, while the converting constructor binds a `const Dog&`. Per [\[over.ics.rank\]/3.2.6](#), the less cv-qualified reference wins. Note that if the constructor and the conversion operator cannot be distinguished by their implicit conversion sequences, such as if `operator Cat` above were made `const`-qualified, both Clang and GCC have a tie-breaker rule that prefers the constructor.

GCC and Clang have been doing this for a long time:

- GCC chooses the conversion function over the constructor since version 7.1 as long as the language mode is set to C++17 or higher. (Version 7.1, released in 2017, is the first version that is reported to support P0135.) See commit 36cbfd, which predates the implementation of GCC's current strategy of tweaking overload resolution priority.

— Clang began choosing the conversion function over the constructor since version 6.0.0, which was released in 2018, as long as the language mode is set to C++17 or higher. See commit 67ef14.

It's possible for the Clang/GCC behavior to cause problems for users who expect direct-initialization to always call a constructor. They might try to implement the conversion function as follows:

```
Dog::operator Cat() {  
    return Cat(*this); // OK in current C++ and MSVC; infinite recursion in Clang and GCC  
}
```

Either the constructor or the conversion function might be instantiated from a template rather than written directly to convert to/from a single type; typically, the constructor. For this reason, Clang's tie-breaker rule of preferring a constructor over a conversion function had to be moved ahead of the template tie-breakers (otherwise, a non-template conversion function would win over a constructor template and would call itself recursively); see commit [5173136](#). Even still, if the constructor template takes its argument by const reference and the conversion function is not const-qualified, the conversion function will win like in Example 4.

At the March 3, 2023 CWG telecon, some implementers expressed concerns over changing the behavior of Example 4, citing domains in which having both a constructor and a conversion function like in Example 4 are actually common. The consensus was that the behavior of Example 4 should not be changed, which rules out the current Clang and GCC approaches.

A related implementation divergence between Clang/GCC and EDG/MSVC that was reported on Stack Overflow involves a binding of a base class reference to a derived class object, and can be illustrated as follows:

```
// Example 5  
struct A1 {};  
  
struct A2 {  
    A2(const A1&); // EDG and MSVC call this (conform to current standard)  
    A2(const A2&);  
};  
  
struct B : A1 {  
    operator A2(); // Clang and GCC call this  
};  
  
A2 a(B{});
```

Conversion functions to both A and “reference to A”

Consider the following:

```
// Example 6  
struct T {  
    T(T const&);  
};  
  
struct S {  
    operator T();  
    operator T&();  
};  
  
S s;  
T t(s);
```

The status quo is this will call `S::operator T&` because the implicit conversion from `S` to `T const&` prefers to bind the reference to an lvalue rather than materializing a temporary ([\[dcl.init.ref\]/5.1.2](#)). EDG, GCC, and MSVC all retain this behavior; Clang's approach of treating `S::operator T` as a separate top-level candidate results in it winning, and `S::operator T&` is not used.

My feeling is that people who write this kind of code are asking for trouble; having both `operator const T&` and `operator T&&` in the same class would be fine (sort of like how we often overload on such pairs of types) but having one return by reference and the other by value should be expected to cause difficulties. If we end up changing the behavior of code like this in order to make the more common cases work properly, the amount of breakage it would cause is likely to be limited. Nevertheless, this example is based on a real Stack Overflow question, so I felt it was worth mentioning.

Hypothetical examples where Clang/GCC might introduce new ambiguities

There were no reported cases on Stack Overflow where Clang's approach introduced an ambiguity that is not present in the current standard. However, it is possible to construct one by having multiple conversion functions that take precedence over all constructors but are ambiguous among themselves:

```
// Example 7
struct Y;

struct X {
    X(const Y&);
};

struct A {
    operator X();
};

struct B {
    operator X();
};

struct Y : A, B { };

X x(Y{}); // well-formed in current C++, ambiguous in Clang
```

My sense is that the amount of real code that would exhibit this kind of breakage is low, although there are approaches we could take to avoid them if CWG considers it important to do so. It was also pointed out on the reflector that users who would encounter such an ambiguity as a result of adopting the Clang approach would have an easy fix: add the constructor `X::X(Y&&)`.

I don't think GCC's algorithm ever introduces new ambiguities. In cases like the above where it is ambiguous which conversion function would be used in order to call the copy/move constructor of the destination class, the overload resolution priority of that constructor is unaffected relative to the status quo; in the specific example above, that means the move constructor of `X` loses in overload resolution, just as in the status quo. If GCC does decide to replace one or more copy/move constructors with a conversion function for the purposes of overload resolution, then all such constructors will use the same conversion function since the reference binding in all cases will use [\[dcl.init.ref\]/5.4.1](#). If that conversion function wins the top-level overload resolution, it is called (there is no need to pick one of the copy/move constructors that originally required it, since the constructor wouldn't be called anyway). The conversion function will never be ambiguous relative to another constructor, because constructors are preferred over conversion functions in case of a tie.

Philosophical difference between Clang and GCC

The Clang approach makes direct-initialization conceptually more similar to copy-initialization in that constructors and conversion functions are both considered when enumerating candidates. This might be viewed as a simplification of the language (although not one that is particularly likely to result in a simplification of the wording). The GCC approach preserves the current philosophy in which constructors are primary for direct-initialization; a conversion function only gets considered if it chosen by a constructor. This difference is not just philosophical; in addition to the divergences already discussed, we can observe that the Clang approach permits copy elision even in cases where no copy constructor, not even a deleted one, has its constraints satisfied:

```
// Example 8
```

```

template <int i = 0>
class NonCopyable {
public:
    NonCopyable(const NonCopyable&) requires(i != 0);

private:
    NonCopyable(int x);
    friend struct Source;
};

struct Source {
    operator NonCopyable<0>();
};

NonCopyable<0> nc(Source{}); // OK in Clang; ill-formed in GCC, MSVC, and the current standard

```

In the March 3, 2023 telecon, CWG appeared to be divided as to how this example should behave. It was pointed out that in current C++, this initialization is well-formed as a copy-initialization, and some members expressed dissatisfaction that it does not also work as a direct-initialization. Others pointed out that the direct-initialization syntax looks as if it connotes a constructor call, and while we all agree that we intend to elide that call under some circumstances (see Example 1), perhaps we should not actually be trying to support Example 8.

I note that the link between direct-initialization and calling constructors already went out the window in C++20 with the introduction of direct-non-list-initialization for aggregates, so if making Example 8 work is judged to be the most useful behavior, then we shouldn't let any perceived link between direct-initialization and calling a constructor hold us back from standardizing such behavior.

Mysterious divergence between EDG and MSVC

The following is very similar to Example 2, but shows that EDG and MSVC do not take the exact same approach:

```

// Example 9
struct Cat {
    Cat(const Cat&);
    Cat(int);
};

struct Dog {
    operator Cat();
    operator int();
};

Cat cat(Dog{}); // ambiguous in current C++

```

Example 2 has both an implicitly declared copy constructor and an implicitly declared move constructor, while Example 9 has a user-declared copy constructor and, therefore, no move constructor. As in Example 2, Clang and GCC will call `operator Cat` in Example 9, and as in Example 2, Example 9 is ambiguous for EDG. However, MSVC, which considers Example 2 ambiguous, accepts Example 9, following Clang and GCC in calling `operator Cat`. It seems that when a *copy* constructor is present, MSVC prefers it over `Cat::Cat(int)`, but when a *move* constructor is present together with the copy constructor, it takes precedence over the copy constructor but does *not* take precedence over `Cat::Cat(int)`. This seems difficult to explain and may simply be a bug in MSVC. (This bug may be unrelated to copy elision, since it appears that older versions of MSVC, which do not perform copy elision in Example 9, also exhibit the nonconforming behavior of choosing the copy constructor.)

Which constructors may be elided?

The following example involves a constructor that can be selected to create an object from an rvalue of the same type, but is not a move constructor because a function template is never a move constructor:

```

// Example 10

```

```
#include <type_traits>

template <bool has_copy_constructor>
struct Cat {
    Cat();
    Cat(const Cat&) requires has_copy_constructor;
    Cat(Cat&&) requires has_copy_constructor;

    template <class C = Cat>
    Cat(std::type_identity_t<C>&&) = delete;
};

struct Dog {
    operator Cat<false>();
};

Dog d;
Cat<false> c(d); // OK in Clang and NVCC++; ill-formed in GCC, MSVC, and the current standard
```

Here, GCC does not elide the constructor call because the constructor is neither a copy nor move constructor. MSVC does not elide it either. Clang does elide, because `Dog::operator Cat<false>` is a candidate during overload resolution. Perhaps surprisingly, NVCC++ is more liberal than MSVC; it *does* elide the constructor call.

There doesn't seem to be an obvious argument for whether only copy and move constructors should be elidable, or any constructor whose parameter-type-list would make it a copy or move constructor if it were not a template. However, the former is slightly easier to specify, and the code snippet above is unlikely to resemble real code.

Description of proposed resolution

After the CWG telecon on March 3, 2023, and in particular the consensus to continue calling the constructor in Example 4, a number of different feasible approaches still remain. One is to simply adopt the EDG approach, which is guaranteed to have no effect on overload resolution. A second is the EDG approach plus a small fix for Example 8, namely: only if there is no viable constructor for the direct-initialization, consider conversion functions. A third approach, which preserves the behavior of Examples 4 and 5 while retaining the Clang/GCC changes for Examples 2 and 3 and the Clang changes for Example 8, is to consider conversion functions as candidates (as Clang) but give them a handicap during overload resolution: a constructor using a standard conversion sequence beats any conversion function. I'll refer to this as “Clang with constructor preference”.

I think Clang with constructor preference is probably the best approach from an evolutionary perspective and the one I would advocate if the author of P0135 were still in the process of revising it. However, after CWG initially appeared to favor the current Clang approach and then had to change course after considering the unexpected consequences for Example 4, there does not appear to be consensus on how to change the overload resolution rules to take conversion functions into account during direct-initialization. It therefore seems to me that we should simply adopt the most conservative (EDG) approach for resolving issue 2327 (with a small tweak, namely that only copy and move constructors may be elided; see Example 10 above), and defer any decision on changing the overload resolution rules to a future point in time (and perhaps a different room, *i.e.*, EWG). Taking the conservative approach here will not close the door to future changes in overload resolution to require elision in more cases.

In any case, the resolution should also cover the (very similar) case where an object of class type is list-initialized from an initializer list with a single element of class type. Since I propose the conservative approach here for parenthesized initialization, I also propose the same approach for list-initialization.

Wording

Wording is relative to [N4944](#).

Edit [basic.def.odr]/4:

A function is *named* by an expression or conversion as follows:

- A function F is named by an expression or conversion if ~~it~~ F is the selected member of an overload set ([basic.lookup], [over.match], [over.over]) in an overload resolution performed as part of forming that expression or conversion, unless ~~it is a pure virtual function and either the expression is not an id-expression naming the function with an explicitly qualified name or the expression forms a pointer to member ([expr.unary.op]):~~
 - F is a pure virtual function and either the expression is not an id-expression naming F with an explicitly qualified name or the expression forms a pointer to member ([expr.unary.op]), or
 - the context is the initialization of an object of class type, where F is the constructor that was selected by overload resolution, and the object is initialized by calling a conversion function instead of calling F ([dcl.init.general], [dcl.init.list]).
- [Note 2: This covers taking the address of functions ([conv.func], [expr.unary.op]), calls to named functions ([expr.call]), operator overloading ([over]), user-defined conversions ([class.conv.fct]), allocation functions for *new-expressions* ([expr.new]), as well as non-default initialization ([dcl.init]). A constructor selected to copy or move an object of class type is considered to be named by an expression or conversion even if the call is actually elided by the implementation ([class.copy.elision]). —end note]
- [...]

Drafting note: I avoid using the term “elided” to describe the form of elision that is the subject of this proposal, in order to avoid having to distinguish between it and elision under [class.copy.elision].

Edit [dcl.init.general]/16.6.2:

Otherwise, if the initialization is direct-initialization, or if it is copy-initialization where the cv-unqualified version of the source type is the same class as, or a derived class of, the class of the destination, constructors are considered. The applicable constructors are enumerated ([over.match.ctor]), and the best one is chosen through overload resolution ([over.match]). Then:

- If overload resolution is successful, ~~the selected constructor is called to initialize the object, with the initializer expression or expression-list as its argument(s):~~
 - If the selected constructor is a copy or move constructor ([class.copy.ctor]) for the class of the destination, the initializer is a parenthesized *expression-list* containing a single element, and the implicit conversion sequence ([over.best.ics.general]) for the constructor's first parameter would bind that parameter to the result object of a conversion function whose return type, ignoring cv-qualification, is the class of the destination, that conversion function is called to initialize the object. The selection of the constructor by the overload resolution performed under this paragraph does not cause it to be named ([basic.def.odr]) or referred to ([dcl.fct.def.delete]) by the initialization.
[Note -?: Therefore, the selection of the constructor to initialize the destination object does not odr-use the constructor, nor require it to be accessible ([class.access]) in the context of the initialization. —end note]
[Example -?:

```
struct Cat {  
    Cat() = default;  
    private:  
    Cat(const Cat&) = delete;  
    Cat(Cat&&) = delete;  
};  
  
struct Dog { operator Cat(); };  
  
Dog d;  
Cat c(d); // OK; calls Dog::operator Cat
```


—end example]

[Note -?: This cannot occur in the context of copy-non-list-initialization, because user-defined conversions are not considered for the first parameter ([over.best.ics.general]). —end note]

- Otherwise, the selected constructor is called to initialize the object, with the initializer expression or *expression-list* as its argument(s).
- Otherwise, if no constructor is viable, the destination type is an aggregate class, and the initializer is a parenthesized *expression-list*, the object is initialized as follows. [...]
- Otherwise, the initialization is ill-formed.

Edit [dcl.init.list]/3.7:

Otherwise, if T is a class type, constructors are considered. The applicable constructors are enumerated and the best one is chosen through overload resolution ([over.match], [over.match.list]). If a narrowing conversion (see below) is required to convert any of the arguments, the program is ill-formed. Then:

- If the selected constructor is a copy or move constructor ([class.copy.ctor]) for T , the *braced-init-list* contains a single element, and the implicit conversion sequence ([over.best.ics.general]) for the constructor's first parameter would bind that parameter to the result object of a conversion function whose return type is T (ignoring cv-qualification), that conversion function is called to initialize the object. The selection of the constructor by the overload resolution performed under this paragraph does not cause it to be named ([basic.def.odr]) or referred to ([dcl.fct.def.delete]) by the initialization.

[Note -?: Therefore, the selection of the constructor to initialize the destination object does not odr-use the constructor, nor require it to be accessible ([class.access]) in the context of the initialization. —end note]

[Example -?:

```
struct Cat {  
    Cat() = default;  
private:  
    Cat(const Cat&) = delete;  
    Cat(Cat&&) = delete;  
};  
  
struct Dog { operator Cat(); };  
  
Dog d;  
Cat c = {d}; // OK; calls Dog::operator Cat  
Cat c{d}; // ditto
```

—end example]

- Otherwise, the selected constructor is called to initialize the object, with the argument(s) specified by [over.match.list].

Drafting note: A function is *needed for constant evaluation* under [expr.const]/20.6 only if it is actually named by an expression (that is potentially constant-evaluated), so the exemption from being named by the wording above will also provide an exemption from being needed for constant evaluation (implying that a copy or move constructor that is defaulted on its first declaration will not be implicitly defined when it is elided as specified above).

Add a note to [over.ics.list]/7.2:

Otherwise, if the parameter is a non-aggregate class x and overload resolution per [over.match.list] chooses a single best constructor c of x to perform the initialization of an object of type x from the argument initializer list:

- If `C` is not an initializer-list constructor and the initializer list has a single element of type `cv U`, where `U` is `X` or a class derived from `X`, the implicit conversion sequence has Exact Match rank if `U` is `X`, or Conversion rank if `U` is derived from `X`.
- Otherwise, the implicit conversion sequence is a user-defined conversion sequence whose second standard conversion sequence is an identity conversion.
[Note -?: Even if `C` would not actually be called to initialize the parameter because a conversion function would be called instead as specified in [dcl.init.list], `C` is still considered to be chosen for the purposes of this paragraph. —end note]

Drafting note: If the wording of [over.ics.list]/7.2 is clarified as suggested in [reflector message 14183](#), the note is still applicable. Also, it's not clear to me that it ever makes a difference whether we consider the user-defined conversion in the user-defined conversion sequence to be `C` or the conversion function, so I've added this note just to be safe.